

DAY 22

APIs for AI Automation

REST Fundamentals, Authentication, JSON, Reliability, Python, and n8n Integration

Course: GPT + AI Agents + RAG + n8n + API Integration + Business Automation

Lecture Duration	60 minutes
Level	Beginner to Intermediate
Week	Week 4 - APIs + n8n + Business Automation
Mini Project	AI Lead Enrichment API Workflow
Primary Outcome	Connect AI workflows to real business systems safely

Day 22 is the bridge between AI reasoning and external business systems. Students learn how software talks to software through APIs, how to authenticate safely, how to handle errors, and how to turn API responses into reliable AI automation.

Detailed lecture handout, lab guide, quiz, homework, and portfolio blueprint.

Table of Contents

1. Day 22 Mission and 60-Minute Lecture Plan	5
Learning objectives	5
2. From RAG Knowledge to Live Business Systems	6
Why APIs matter to AI agents	6
3. What Is an API?	7
Core vocabulary	7
4. HTTP Request Anatomy	8
Five pieces to inspect	8
URL anatomy	8
5. HTTP Methods: GET, POST, PUT, PATCH, DELETE	9
Read vs write risk	9
6. JSON: The Language of Business APIs	10
Important JSON types	10
API response != automatically trusted for every purpose	10
7. Headers and Content Types	11
Content-Type example	11
8. Authentication: Proving Who the Client Is	12
Secrets management rules	12
9. Status Codes: Understanding API Outcomes	13
10. Error Handling and Retry Strategy	14
Retryable vs non-retryable examples	14
Exponential backoff concept	14
11. Rate Limits and Quotas	15
Design responses to rate limiting	15
n8n reliability note	15
12. Pagination: When One Response Is Not Enough	16
Safe pagination loop	16
13. Query Parameters, Filtering, Sorting, and Field Selection	17
Agent design lesson	17

14. Idempotency and Duplicate Side Effects	18
Idempotency concept	18
AI-agent relevance	18
15. API Contracts and Schema Validation	19
Three validation layers	19
16. Python API Lab: GET a Resource	20
Why `timeout` matters	20
17. Python API Lab: POST Structured Data	21
Use `json=payload` for JSON APIs	21
18. Build a Reusable API Client Layer	22
Benefits	22
19. APIs as AI Agent Tools	23
Bad vs good tool design	23
20. Normalized Tool Results	24
Failure shape	24
21. API Security for AI Automation	25
Threats to consider	25
Controls	25
22. API Observability and Logging	26
Useful log fields	26
23. API vs Webhook	27
24. n8n HTTP Request Node - Mental Model	28
Configuration checklist	28
25. n8n Mapping Example: Retrieve a Lead	29
Expressions	29
26. Mini Project - AI Lead Enrichment API Workflow	30
Input	30
27. Mini Project - Data Contracts and Scoring	31
Normalized enrichment output	31
AI interpretation output	31
Deterministic scoring example	31

28. Mini Project - Complete Output and Failure Paths	32
CRM-ready result	32
Failure cases to test	32
Expected behavior	32
29. Classroom Exercise - Read an API Documentation Page	33
30. Day 22 Quiz	34
31. Homework	35
Assignment 1 - API anatomy	35
Assignment 2 - Python client	35
Assignment 3 - Error matrix	35
Assignment 4 - n8n mapping	35
Assignment 5 - Portfolio mini project	35
32. Interview and Upwork Preparation	36
Common interview questions	36
Client discovery questions	36
33. Day 22 Golden Rules and Final Checklist	37
Student completion checklist	37
34. Preview - Day 23: Webhooks and Real-Time Automation	38
Day 23 topics	38
35. References and Further Reading	39
HTTP fundamentals	39
n8n	39

1. Day 22 Mission and 60-Minute Lecture Plan

The previous week focused on RAG: how AI retrieves trusted knowledge. Week 4 focuses on connecting AI to live systems and automating real business processes. Day 22 introduces the API concepts needed before n8n and webhook automation.

Day 22 goal: Understand how an AI application can safely read from and write to external systems using HTTP APIs, then build a small integration that retrieves business data and passes clean structured results to an AI workflow.

Time	Topic
0-5 min	Review Week 3 and define API integration
5-12 min	API, client/server, endpoint, request, response
12-20 min	HTTP methods, URL parts, headers, JSON
20-28 min	Authentication and secrets
28-36 min	Status codes, errors, retries, rate limits
36-43 min	Pagination, filtering, idempotency
43-50 min	Python requests lab
50-56 min	n8n HTTP Request mapping
56-60 min	Mini project, quiz, homework

Learning objectives

- Explain what an API is and why AI automation depends on APIs.
- Understand client/server communication and endpoints.
- Use GET, POST, PUT, PATCH, and DELETE appropriately at a beginner level.
- Read request/response headers and JSON bodies.
- Recognize common authentication patterns.
- Interpret status codes and design safe retry behavior.
- Understand rate limits, pagination, filtering, and idempotency.
- Call a REST API from Python.
- Map the same request into an n8n HTTP Request node.
- Design an API integration layer that keeps secrets and hard business rules outside the LLM.

2. From RAG Knowledge to Live Business Systems

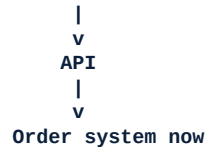
RAG answers questions from documents. Many business tasks require current transactional data or actions. APIs are the standard bridge between an AI workflow and systems such as CRM, ecommerce, support, calendars, finance platforms, and internal services.

Knowledge vs live systems

Question: What is our refund policy?



Question: Has order ORD-1002 shipped?



Why APIs matter to AI agents

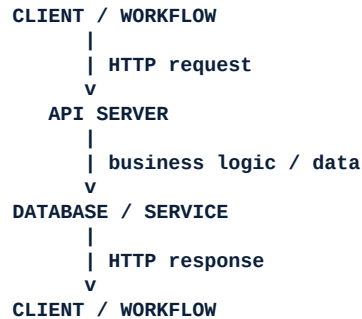
- LLMs do not automatically know private CRM or ERP data.
- Dynamic facts can change after the model was trained or after a conversation began.
- Business actions must be executed by software, not imagined by the model.
- APIs provide structured boundaries for retrieval and action.

Golden rule: The model can decide or recommend. The application calls the API. The external system remains the source of truth.

3. What Is an API?

API stands for Application Programming Interface. In this course, think of an API as a defined contract that lets one software system request data or actions from another software system.

Basic client/server exchange



Core vocabulary

Term	Meaning	Example
Client	System sending the request	Python script, n8n workflow
Server	System receiving the request	CRM API
Endpoint	Specific API address for a resource/action	/v1/customers/123
Request	Message sent to server	GET customer 123
Response	Server answer	200 + customer JSON
Resource	Business object	customer, order, ticket
Payload	Data sent in request body	new lead fields
Schema	Expected data shape	name:string, score:number

4. HTTP Request Anatomy

Most modern business APIs are accessed over HTTP. A request combines a method, URL, headers, optional query parameters, and sometimes a body.

```
POST https://api.example.com/v1/leads?notify=true
Authorization: Bearer <TOKEN>
Content-Type: application/json
```

```
{
  "name": "Acme Corp",
  "email": "buyer@example.com",
  "source": "website"
}
```

Five pieces to inspect

Piece	Question to ask
Method	What operation is requested?
URL / endpoint	Which system and resource?
Query parameters	What filtering/options are requested?
Headers	What metadata/authentication is required?
Body	What data is being sent?

URL anatomy

```
https://api.example.com/v1/orders/ORD-1002?include=tracking
|-----| |-----| |-----| |-----|
scheme      host      path      query
```


5. HTTP Methods: GET, POST, PUT, PATCH, DELETE

HTTP methods communicate the intended operation. Exact semantics depend on the API, so always read the provider documentation.

Method	Typical intent	Body?	Example
GET	Read/retrieve	Usually no	Get order details
POST	Create/trigger	Usually yes	Create lead
PUT	Replace resource	Yes	Replace customer profile
PATCH	Partially update	Yes	Change lead status
DELETE	Delete resource	Usually no/small	Delete draft record

Read vs write risk

Risk progression

GET / search / list

|

v

Low-risk read operations

|

v

POST / PATCH / PUT / DELETE

|

v

Writes, side effects, approvals, validation

For AI agents, start with read-only API access whenever possible. Add write actions only after validation, permission checks, logging, and approval rules are defined.

6. JSON: The Language of Business APIs

JSON is a common structured format for API requests and responses. Students already used JSON for structured AI outputs; the same skill transfers directly to API integration.

```
{
  "lead_id": "L-9001",
  "company": "Acme Corp",
  "employees": 240,
  "qualified": true,
  "tags": ["enterprise", "inbound"],
  "owner": {
    "name": "Maya",
    "team": "sales"
  }
}
```

Important JSON types

Type	Example
String	"Acme Corp"
Number	240
Boolean	true
Null	null
Array	["enterprise", "inbound"]
Object	{"name": "Maya"}

API response != automatically trusted for every purpose

A response should still be validated for expected fields, types, authorization scope, and business rules before it controls a consequential action.

7. Headers and Content Types

Headers carry additional request or response metadata. HTTP headers can communicate authentication, content type, caching, rate-limit details, request identifiers, and other protocol information.

Header	Common purpose
Authorization	Credentials/token
Content-Type	Media type of request body
Accept	Preferred response media type
User-Agent	Client identification
Idempotency-Key	Deduplicate supported write requests
Retry-After	Server guidance about retry timing, when provided
X-Request-ID	Trace/correlation identifier in some APIs

Content-Type example

`Content-Type: application/json`

This tells the receiver that the body is JSON. Different APIs may accept form data, multipart uploads, binary content, or other media types.

8. Authentication: Proving Who the Client Is

Most useful business APIs require authentication. The exact mechanism is provider-specific.

Pattern	Typical shape	Notes
API key header	X-API-Key: ...	Simple provider-issued secret
Bearer token	Authorization: Bearer ...	Common token pattern
Basic auth	Authorization header	Username/password-style credentials
OAuth 2.0	Access token + scopes	Delegated access and refresh flows
Signed request	Signature + timestamp	Used by some financial/cloud APIs

Secrets management rules

- Never place production secrets in prompts.
- Do not hardcode secrets in source control.
- Use environment variables, secret managers, or n8n credentials.
- Request the minimum scopes/permissions needed.
- Rotate credentials and remove unused access.
- Never log full access tokens.

```
# Python example
import os

API_TOKEN = os.environ["CRM_API_TOKEN"]
headers = {
    "Authorization": f"Bearer {API_TOKEN}",
    "Accept": "application/json",
}
```

9. Status Codes: Understanding API Outcomes

HTTP response status codes indicate the outcome class of a request. They are grouped into 1xx informational, 2xx success, 3xx redirection, 4xx client errors, and 5xx server errors.

Code	Meaning	Typical automation response
200	OK	Process result
201	Created	Store new resource ID
204	No Content	Treat as success without JSON body
400	Bad Request	Fix request; do not blindly retry
401	Unauthenticated/invalid auth	Refresh/fix credentials
403	Forbidden	Stop or escalate permission issue
404	Not Found	Ask/check identifier
409	Conflict	Inspect state/idempotency
422	Validation problem in many APIs	Fix data
429	Too Many Requests	Backoff/retry per provider guidance
500	Server error	Limited retry
503	Service unavailable	Retry with backoff

Do not treat every non-200 response as the same error. Reliable automation classifies failures and chooses retry, correction, escalation, or stop behavior.

10. Error Handling and Retry Strategy

Classify before retry

```

API failure
  |
  v
Temporary? ---- yes ----> bounded retry + backoff
  | no
  v
Bad data? ----- yes ----> correct / ask user
  | no
  v
Permission? --- yes ----> stop / escalate
  | no
  v
Unknown -----> log + safe failure

```

Retryable vs non-retryable examples

Condition	Retry?	Why
Timeout	Usually yes, limited	May be transient
429 rate limit	Yes, delayed	Request volume too high
503 unavailable	Usually yes	Service may recover
400 malformed JSON	No	Same request will fail again
401 invalid token	Not unchanged	Credentials must change/refresh
404 wrong order ID	No	Need different input

Exponential backoff concept

```

attempt 1 -> wait 1s
attempt 2 -> wait 2s
attempt 3 -> wait 4s
then fail safely / escalate

```

Production systems often add jitter and honor provider-specific retry headers or SDK behavior.

11. Rate Limits and Quotas

APIs commonly limit request volume to protect systems and enforce account plans. Limits can be measured per second, minute, day, user, token, endpoint, or another provider-defined dimension.

Design responses to rate limiting

- Avoid unnecessary duplicate requests.
- Cache stable data where appropriate.
- Batch requests when the API supports it.
- Use bounded retry/backoff for 429 responses.
- Queue non-urgent work.
- Monitor remaining quota when the provider exposes it.

n8n reliability note

n8n documents a Retry On Fail setting for nodes and recommends batching/waiting patterns for rate-limit handling. The exact workflow should follow the target API limits.

Rate-limited workflow



12. Pagination: When One Response Is Not Enough

List endpoints often return only part of a dataset. Pagination requires the client to request the next page until the desired stopping condition is reached.

Pattern	Example concept
Page number	?page=2&limit;=100
Offset	?offset=100&limit;=100
Cursor	?cursor=abc123
Next URL	response contains next link

Safe pagination loop

```
results = []
cursor = None

while True:
    page = fetch_page(cursor=cursor)
    results.extend(page["items"])

    cursor = page.get("next_cursor")
    if not cursor:
        break
```

Always define a maximum page/item budget in production. A bug or unexpected API response should not create an infinite data-fetch loop.

13. Query Parameters, Filtering, Sorting, and Field Selection

Good API integrations ask the server for only the data they need. This reduces latency, cost, payload size, and unnecessary exposure.

```
GET /v1/leads?status=open&country=US&limit=50&sort=-created_at
```

Parameter concept	Business use
filter	Only open tickets / active customers
limit	Control result count
sort	Newest first / highest priority
fields	Return selected fields if supported
include / expand	Include related records if supported

Agent design lesson

Do not give an LLM unlimited freedom to invent arbitrary API filters for sensitive systems. Validate allowed values and build safe tool schemas.

14. Idempotency and Duplicate Side Effects

A network timeout creates ambiguity: the server may have processed a write even if the client never received the response. Repeating the request can accidentally create duplicates.

Duplicate risk

```
POST create_invoice
  |
  v
server creates invoice
  |
  v
network timeout
  |
  v
client retries POST
  |
  v
second invoice?
```

Idempotency concept

Where the provider supports it, an idempotency key or naturally idempotent operation can let repeated requests represent the same intended action instead of creating multiple side effects.

Idempotency-Key: order-ORD-1002-refund-request-1

AI-agent relevance

- Never let an agent repeatedly call a write tool because the response was unclear.
- Track tool call IDs and business transaction IDs.
- Use approval and deduplication for payments, refunds, emails, and record creation.

15. API Contracts and Schema Validation

An API integration is a contract between systems. Both request and response shapes should be validated.

Expected lead response:

```
{
  "id": "L-1001",
  "company": "Acme",
  "status": "open",
  "score": 72
}
```

Business expectation:

- id must exist
- status must be an allowed enum
- score must be 0..100

Three validation layers

- Transport validation: did HTTP succeed?
- Schema validation: does the JSON have expected fields/types?
- Business validation: are values allowed and appropriate for this operation?

Validation pipeline

```
HTTP response
  |
  v
status check
  |
  v
JSON parse
  |
  v
schema validation
  |
  v
business rules
  |
  v
safe downstream use
```

16. Python API Lab: GET a Resource

The following pattern demonstrates a read request with timeout, status handling, and structured return. Replace the example endpoint with a real documented API during practice.

```
import os
import requests

BASE_URL = "https://api.example.com/v1"
TOKEN = os.environ["EXAMPLE_API_TOKEN"]

def get_lead(lead_id: str):
    url = f"{BASE_URL}/leads/{lead_id}"
    headers = {
        "Authorization": f"Bearer {TOKEN}",
        "Accept": "application/json",
    }

    response = requests.get(
        url,
        headers=headers,
        timeout=15,
    )

    if response.status_code == 404:
        return {"success": False, "error": "LEAD_NOT_FOUND"}

    response.raise_for_status()
    data = response.json()

    return {
        "success": True,
        "data": data,
    }
```

Why `timeout` matters

Without timeouts, a workflow can wait much longer than intended when a remote system stalls. Production code should define connect/read timeout behavior appropriate to the operation.

17. Python API Lab: POST Structured Data

```
import requests

def create_lead(payload: dict):
    response = requests.post(
        f"{BASE_URL}/leads",
        headers={
            "Authorization": f"Bearer {TOKEN}",
            "Content-Type": "application/json",
        },
        json=payload,
        timeout=15,
    )

    if response.status_code == 400:
        return {
            "success": False,
            "error": "INVALID_REQUEST",
        }

    response.raise_for_status()
    return {
        "success": True,
        "data": response.json(),
    }

lead = create_lead({
    "company": "Acme Corp",
    "email": "buyer@example.com",
    "source": "website"
})
```

Use `json=payload` for JSON APIs

The Python requests library serializes the dictionary and sets an appropriate JSON content type when using the `json=` argument. Always follow the target API documentation.

18. Build a Reusable API Client Layer

Avoid scattering raw HTTP calls throughout an agent application. Centralize authentication, timeouts, error mapping, logging, and response validation.

```
class CRMClient:
    def __init__(self, base_url, token):
        self.base_url = base_url.rstrip("/")
        self.session = requests.Session()
        self.session.headers.update({
            "Authorization": f"Bearer {token}",
            "Accept": "application/json",
        })

    def get_lead(self, lead_id):
        r = self.session.get(
            f"{self.base_url}/leads/{lead_id}",
            timeout=15,
        )
        return normalize_response(r)

    def update_lead(self, lead_id, fields):
        r = self.session.patch(
            f"{self.base_url}/leads/{lead_id}",
            json=fields,
            timeout=15,
        )
        return normalize_response(r)
```

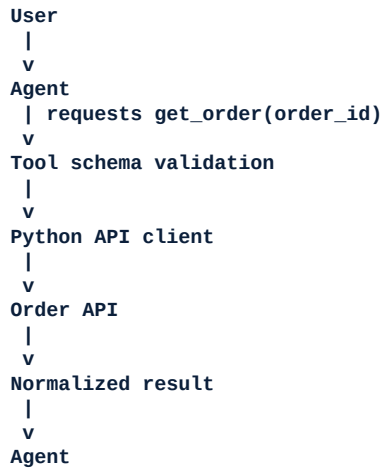
Benefits

- One place for credentials.
- One place for retry policy.
- Consistent error codes for the agent.
- Easier testing/mocking.
- Easier migration if the provider changes.

19. APIs as AI Agent Tools

The safest pattern is to wrap an API behind a narrow application tool instead of giving the model raw HTTP freedom.

Agent tool boundary



Bad vs good tool design

Bad	Better
<code>generic_http_request(url, method, body)</code>	<code>get_order(order_id)</code>
<code>execute_any_sql(query)</code>	<code>find_customer(email)</code>
<code>call_crm(endpoint, payload)</code>	<code>create_lead(company,email,source)</code>

Narrow tools reduce prompt-injection risk, simplify authorization, and make the agent easier to evaluate.

20. Normalized Tool Results

External APIs use different response formats. Agents work more reliably when the application converts provider-specific responses into stable internal result shapes.

```
{
  "success": true,
  "data": {
    "lead_id": "L-9001",
    "status": "open",
    "company": "Acme Corp"
  },
  "error": null,
  "meta": {
    "source": "crm",
    "request_id": "req_abc"
  }
}
```

Failure shape

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "RATE_LIMITED",
    "message": "CRM request limit reached",
    "retryable": true
  }
}
```

The model can then reason about a small stable vocabulary instead of every provider-specific error message.

21. API Security for AI Automation

Threats to consider

- Prompt injection attempting to change tool arguments or targets.
- User-provided identifiers referencing another tenant/customer.
- Secrets leaked into model context or logs.
- Over-scoped credentials.
- Unvalidated write payloads.
- Server-side request forgery risk from unrestricted URL tools.
- Replay or duplicate write operations.

Controls

Risk	Control
Cross-customer access	Bind trusted authenticated customer ID in application context
Arbitrary endpoints	Allowlisted tools/endpoints
Secret exposure	Credential store/environment variables
Dangerous writes	Validation + permission + approval
Duplicates	Idempotency/deduplication
Injection	Treat user text as untrusted; narrow schemas
Auditability	Correlation IDs and structured logs

22. API Observability and Logging

When an API automation fails, the team should be able to determine where and why without logging sensitive credentials.

Useful log fields

- workflow/run ID
- tool name / endpoint category
- HTTP method
- status code
- duration
- retry count
- provider request ID
- normalized error code
- business object ID where safe

```
{  
  "workflow_id": "lead-enrichment-42",  
  "tool": "get_company_profile",  
  "status": 200,  
  "duration_ms": 384,  
  "retry_count": 0,  
  "provider_request_id": "req_abc123"  
}
```

Do not log Authorization headers, API tokens, passwords, full payment details, or other secrets.

23. API vs Webhook

Students often confuse APIs with webhooks. APIs are typically used when your system initiates a request. Webhooks are typically used when another system pushes an event to your endpoint.

API request	Webhook event
You ask for data/action	External system notifies you
Usually client-initiated	Event-initiated
Example: get order	Example: order shipped
Polling possible	Push model
Day 22 focus	Day 23 focus

Pull vs push

API pull

n8n ---> GET order ---> Store

Webhook push

Store ---> order.shipped ---> n8n

Day 23 will build on today by turning n8n into an HTTP endpoint that receives real-time events.

24. n8n HTTP Request Node - Mental Model

The n8n HTTP Request node can call REST APIs without writing Python. The same concepts still apply: method, URL, authentication, headers, query parameters, request body, response handling, retries, and pagination.

n8n API workflow

```
Trigger
|
v
Set / Edit Fields
|
v
HTTP Request
|
v
IF / validation
|
+--> success --> AI / database / CRM
|
+--> failure --> retry / error flow
```

Configuration checklist

- Choose method.
- Enter endpoint URL.
- Select credentials instead of pasting secrets into fields.
- Add query parameters.
- Send JSON body if required.
- Set response expectations.
- Configure Retry On Fail where appropriate.
- Handle API-specific pagination.
- Route failures explicitly.

25. n8n Mapping Example: Retrieve a Lead

Conceptual configuration for an HTTP Request node:

Setting	Example
Method	GET
URL	https://api.example.com/v1/leads/{{json.lead_id}}
Authentication	Saved credential
Header	Accept: application/json
Response	JSON
Retry On Fail	Enabled only per API policy
Next node	Validate / normalize fields

Expressions

```
{{json.lead_id}}  
{{json.email}}  
{{json.company}}
```

Expressions let node inputs come from earlier workflow data. Validate user-supplied values before using them in sensitive API calls.

26. Mini Project - AI Lead Enrichment API Workflow

Build a beginner portfolio workflow that accepts a lead, retrieves a company profile from a mock/external enrichment API, asks AI to summarize the account, applies deterministic qualification rules, and produces a CRM-ready output.

Project architecture

```
Lead form / JSON
  |
  v
Validate fields
  |
  v
Company Enrichment API
  |
  v
Normalize response
  |
  v
GPT: summarize + classify
  |
  v
Deterministic lead score rules
  |
  v
CRM-ready JSON / n8n
```

Input

```
{
  "company": "Acme Corp",
  "email": "buyer@acme.example",
  "country": "US",
  "message": "We need an AI support solution for 80 agents."
}
```

27. Mini Project - Data Contracts and Scoring

Normalized enrichment output

```
{
  "company": "Acme Corp",
  "domain": "acme.example",
  "industry": "Ecommerce",
  "employees": 240,
  "country": "US",
  "source_verified": true
}
```

AI interpretation output

```
{
  "summary": "Mid-size ecommerce company seeking AI support automation.",
  "intent": "sales_opportunity",
  "use_case": "customer_support_automation",
  "urgency": "medium",
  "missing_information": ["budget", "timeline"]
}
```

Deterministic scoring example

```
score = 0

if employees >= 100:
    score += 30
if intent == "sales_opportunity":
    score += 30
if use_case == "customer_support_automation":
    score += 20
if country in ALLOWED_MARKETS:
    score += 20

qualified = score >= 70
```

AI interprets the text. Code calculates the hard score. The API supplies external business facts. This separation is a core professional pattern.

28. Mini Project - Complete Output and Failure Paths

CRM-ready result

```
{
  "company": "Acme Corp",
  "industry": "Ecommerce",
  "employees": 240,
  "summary": "Mid-size ecommerce company seeking AI support automation.",
  "lead_score": 100,
  "qualified": true,
  "next_action": "sales_review",
  "missing_information": ["budget", "timeline"]
}
```

Failure cases to test

- API token missing.
- Company not found.
- 429 rate limit.
- 503 temporary outage.
- Malformed API JSON.
- Lead email missing.
- AI classifies uncertainly.
- Duplicate lead submitted twice.
- Country outside allowed market.
- Prompt injection embedded inside lead message.

Expected behavior

The workflow should never invent enrichment data when the API fails. It should retain the original lead, expose a normalized error, and route the record to retry, human review, or incomplete-enrichment handling.

29. Classroom Exercise - Read an API Documentation Page

Give students any safe public API documentation page. They must answer the following before writing code.

Question	Student answer
Base URL	
Authentication method	
Endpoint	
HTTP method	
Required path parameters	
Required query parameters	
Required headers	
Request body schema	
Success status code	
Important error codes	
Pagination method	
Rate-limit guidance	
Example response	
Is the operation read-only or side-effecting?	

Professional habit: do not start by guessing code. Start by reading the API contract.

30. Day 22 Quiz

#	Question	Answer
1	What is an API?	A defined software interface/contract
2	Which method commonly retrieves data?	GET
3	Which format is common for REST API bodies?	JSON
4	What does a 2xx code generally mean?	Success class
5	Should a 400 request be blindly retried?	No
6	What does 429 usually indicate?	Rate limit
7	What is pagination?	Fetching a list in multiple pages
8	Where should API secrets be stored?	Credential/secret storage
9	What is idempotency for?	Avoid duplicate side effects on retries
10	Should an LLM call arbitrary URLs directly?	Usually no; use narrow allowlisted tools
11	API vs webhook?	Request initiated vs event pushed
12	Why normalize API errors?	Stable downstream behavior
13	What is a timeout?	Maximum wait boundary for network operation
14	What should own strict thresholds?	Deterministic code/workflow rules
15	What should happen after repeated temporary failures?	Fail safely / escalate

31. Homework

Assignment 1 - API anatomy

Choose one documented API and create a one-page integration specification containing method, endpoint, authentication, headers, parameters, request body, success response, error codes, rate-limit behavior, and pagination.

Assignment 2 - Python client

Write Python functions for one GET endpoint and one POST or PATCH endpoint. Include environment-variable credentials, timeout, status handling, normalized errors, and JSON parsing.

Assignment 3 - Error matrix

Error	Retry	Ask user	Escalate	Stop
400 invalid request				
401 auth				
403 forbidden				
404 not found				
429 rate limit				
500 server error				
timeout				

Assignment 4 - n8n mapping

Re-create your Python GET call using n8n HTTP Request. Use a saved credential, expression-based ID, error route, and normalized output.

Assignment 5 - Portfolio mini project

Complete the AI Lead Enrichment workflow and create screenshots plus a short README explaining business problem, architecture, API, AI responsibility, deterministic rules, failure behavior, and measurable business value.

32. Interview and Upwork Preparation

Common interview questions

Question	Strong answer focus
What is REST API integration?	Client/server request/response using documented endpoints
GET vs POST?	Read vs create/trigger; follow provider semantics
How do you handle 429?	Backoff, retry limits, queues, provider guidance
How do you secure API keys?	Secret storage, least privilege, rotation, no prompt/log exposure
Why idempotency?	Safe retries for side-effecting operations
How do you integrate APIs with AI agents?	Narrow tools + schema validation + app execution
How do you debug failed API workflows?	Status, body, request ID, logs, retry classification
API vs RAG?	Live transactional truth vs document knowledge
API vs webhook?	Pull/request vs pushed event

Client discovery questions

- Which business systems must be integrated?
- Does the vendor have official API documentation?
- Which operations are read-only vs write?
- What authentication and OAuth scopes are required?
- What are the provider rate limits?
- How should errors and retries be handled?
- Which writes require human approval?
- How are duplicates prevented?
- What logs/audit records are required?

33. Day 22 Golden Rules and Final Checklist

- Read the API documentation before writing the integration.
- The application executes APIs; the model does not magically access systems.
- Keep secrets outside prompts and source code.
- Start AI agents with read-only API tools.
- Validate request arguments and response schemas.
- Classify errors before retrying.
- Use bounded retries and timeouts.
- Respect rate limits and pagination.
- Protect write operations with permissions, business rules, and approvals.
- Design for idempotency and duplicate prevention.
- Normalize provider-specific responses for agents.
- Log enough to debug without logging secrets.
- Use APIs for live truth, RAG for knowledge, and webhooks for pushed events.

Day 22 formula:

Reliable AI API Integration = Clear API Contract + Safe Authentication + Structured JSON + Validation + Error Handling + Rate-Limit Control + Narrow Agent Tools + Business Rules + Observability

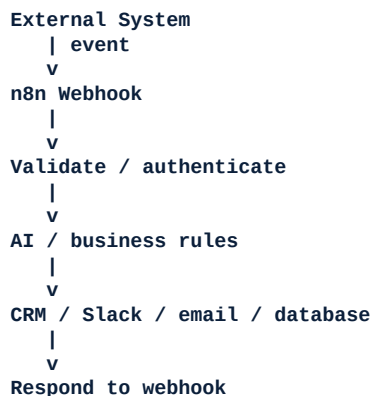
Student completion checklist

- I can explain API/client/server/endpoint.
- I can read an HTTP request.
- I know GET/POST/PUT/PATCH/DELETE basics.
- I understand JSON bodies and headers.
- I know common authentication patterns.
- I can interpret common status codes.
- I can design a retry matrix.
- I understand pagination and rate limits.
- I understand idempotency.
- I can call a REST API in Python.
- I can map a request to n8n HTTP Request.
- I can wrap an API as a narrow AI-agent tool.

34. Preview - Day 23: Webhooks and Real-Time Automation

Day 23 changes the direction of communication. Instead of repeatedly asking another system whether something changed, your workflow will receive events when they happen.

Day 23 architecture



Day 23 topics

- Webhook endpoints and HTTP POST events.
- Webhook payloads and headers.
- Test URL vs production URL in n8n.
- Respond to Webhook.
- Webhook authentication/signatures.
- Idempotency and duplicate events.
- Retries from webhook senders.
- Event-driven AI automation.
- Real-time customer-support and sales examples.

After Day 23, students will understand both sides of integration: APIs for requesting data/actions and webhooks for receiving events.

35. References and Further Reading

The following primary documentation was used to align the lecture with current HTTP and n8n behavior as of August 2026.

HTTP fundamentals

MDN Web Docs - HTTP overview: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>

MDN Web Docs - HTTP request methods: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

MDN Web Docs - HTTP response status codes: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>

MDN Web Docs - HTTP headers: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers>

MDN Web Docs - Content-Type header: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Type>

n8n

n8n Docs - HTTP Request node: <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/>

n8n Docs - Handling API rate limits: <https://docs.n8n.io/integrations/builtin/rate-limits/>

n8n Docs - Error handling: <https://docs.n8n.io/flow-logic/error-handling/>

n8n Docs - Respond to Webhook: <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.respondtowehook/>

Note: API providers can change endpoints, authentication requirements, quotas, and schemas. Always confirm the current official documentation before production deployment.